WEEK 6 SKILL OSSP

Part 1 — Backslash Escaping / Special Character Processing

1. **Created a new folder for week 6 skill**

```
suraj@DESKTOP-13F1P77:~$ mkdir week6_s
suraj@DESKTOP-13F1P77:~$ cd ~/week6_s
suraj@DESKTOP-13F1P77:~/week6_s$ pwd
/home/suraj/week6_s
```

2. Basic Backslash Escaping varshini@DESKTOP-BLT5L3R:~/week6_s$ echo "Hello

```
Hello\nWorld
suraj@DESKTOP-13F1P77:~/week6_s$ echo -e "Hello\nWorld"
Hello
World
```

3. Escaped Spaces

```
suraj@DESKTOP-13F1P77:~/week6_s$ echo Hello\ World
Hello World
suraj@DESKTOP-13F1P77:~/week6_s$ echo Hello World
Hello World
```

4. Escape Special Characters

```
*
suraj@DESKTOP-13F1P77:~/week6_s$ echo \&
&
suraj@DESKTOP-13F1P77:~/week6_s$ echo \\
\
suraj@DESKTOP-13F1P77:~/week6_s$
```

5. Preserve Character

```
suraj@DESKTOP-13F1P77:~/week6_s$ echo '$HOME'
$HOME
suraj@DESKTOP-13F1P77:~/week6_s$ echo "$HOME"
/home/suraj
suraj@DESKTOP-13F1P77:~/week6_s$
```

6. Validate Parser Output

```
suraj@DESKTOP-13F1P77:~/week6_s$ printf '<%s>\n' Hello\ World
<Hello World>
suraj@DESKTOP-13F1P77:~/week6_s$ printf '<%s>\n' "Hello World"
<Hello World>
suraj@DESKTOP-13F1P77:~/week6_s$ printf '<%s>\n' 'Hello World'
<Hello World>
```

7. Test Complex Input

```
suraj@DESKTOP-13F1P77:~/week6_s$ echo "Hello\ World \$USER \& \*"
Hello\ World $USER \& \*
```

Summary:

The experiment demonstrates how backslash escaping is used in Linux shell processing to preserve spaces and special characters. Escape sequences such as \n were tested, along with escaped spaces, shell metacharacters, quoted strings, and complex inputs. The outputs were validated using printf to observe how the shell parses individual arguments. The experiment confirms that escape

PART 2 — Child Processes

1. Creating the Child Process Program, Compiling and Execute

```
suraj@DESKTOP-13F1P77:~/week6_s$ cd ~/week6_s
suraj@DESKTOP-13F1P77:~/week6_s$ nano child_process.c
suraj@DESKTOP-13F1P77:~/week6_s$ gcc child_process.c -o child_process
suraj@DESKTOP-13F1P77:~/week6_s$ ./child_process
Parent Process: PID = 9362
Waiting for child process...
Child Process: PID = 9363
Executing command: ls
child_process  child_process.c  week6_s
Child process completed.
```

2. Passing Arguments to a Child Process

```
suraj@DESKTOP-13F1P77:~/week6_s$ gcc child_process.c -o child_process
suraj@DESKTOP-13F1P77:~/week6_s$ ./child_process
Child Process: PID = 9389
Running: echo Hello OSSP Week 6
Hello OSSP Week 6
Child process completed.
```

3. Final Process Check

```
suraj@DESKTOP-13F1P77:~/week6_s$ ps
    PID TTY          TIME CMD
   9207 pts/2    00:00:00 bash
   9454 pts/2    00:00:00 ps
```

Summary:

The experiment demonstrated backslash escaping and special character processing in Ubuntu. We tested escaped spaces, special symbols, escape sequences, character preservation using quotes, and validated how the shell parses normal and complex inputs using printf. We also implemented process creation and execution in C using fork(), exec(), and wait(). The child process executed commands with arguments, the parent process managed the child, and execution errors were handled using perror().